RAJALAKSHMI ENGINEERING COLLEGE

Thiviyadaran V 251001292-2025-IT

T2

Started on	Thursday, 27 August 2026, 2:31 PM
State	Finished
Completed on	Thursday, 27 August 2026, 2:32 PM
Time taken	1 min 23 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    long diff = *(long *)a - *(long *)b;
    return (diff > 0) - (diff < 0);
}

int main() {
    int n, m;

    if (scanf("%d", &n) != 1) return 0;
    long g[n];
    for (int i = 0; i < n; i++) {
        scanf("%ld", &g[i]);
    }

    if (scanf("%d", &m) != 1) return 0;
    long s[m];
    for (int i = 0; i < m; i++) {
        scanf("%ld", &s[i]);
    }


    qsort(g, n, sizeof(long), compare);
    qsort(s, m, sizeof(long), compare);


    int i = 0;
    int j = 0;
```

```
31
32      while (i < n && j < m) {
33          if (s[j] >= g[i]) {
34              i++;
35          }
36          j++;
37      }
38
39      printf("%d\n", i);
40      return 0;
41  }
```

	Input	Expected	Got	
✔	2 1 2 3 1 2 3	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course